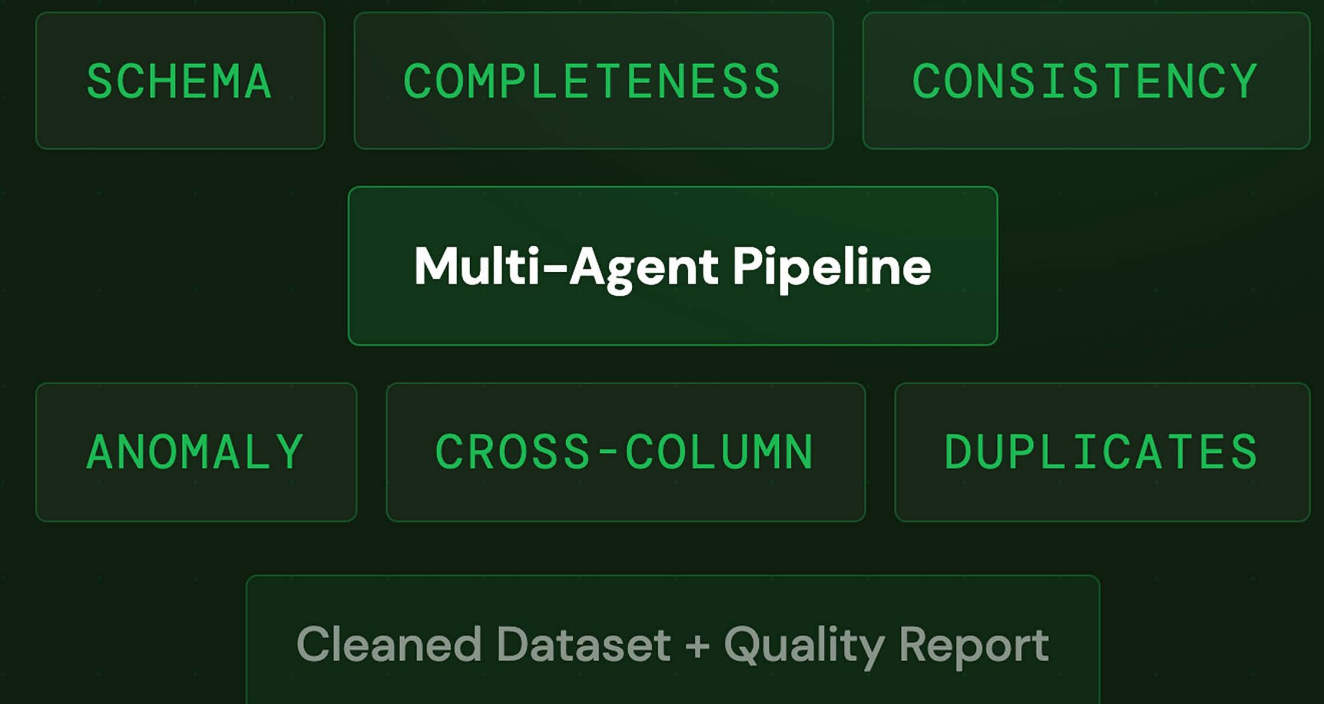


NoiPA Data Quality Copilot

A governed, multi-agent pipeline that automatically diagnoses, cleans, and certifies heterogeneous public-sector datasets.

Michele Turco • Mattia Sebastiani • Sofia Bruni



The Data Quality Problem

Heterogeneous Sources

Public-sector data arrives from multiple systems and formats, CSV, JSON, database extracts, all inconsistent with one another.

01

Hidden Defects

Placeholder tokens, inconsistent date formats, duplicate columns, and semantic conflicts hide behind apparently populated cells.

02

Manual Bottleneck

Repetitive, expensive, hard to document. Manual review does not scale and cannot produce a consistent audit trail across teams.

03

Downstream Risk

Analytics, reporting, and operations silently inherit data quality problems, uncertainty that compounds invisibly over time.

04

The real cost of poor data quality is not only inefficiency, it is **slower decisions and lower institutional trust**.

A Governed Remediation Pipeline

AI reasoning combined with deterministic validation, every step bounded, safe, and fully auditable.

DIAGNOSE

AI-Assisted Detection

Specialized agents inspect each column for type errors, format inconsistencies, completeness gaps, and anomalies. Local Python computes the evidence; agents interpret it.

01

INTERVENE

Governed Automation

Every proposed fix is validated by host-side checks before being applied. Low-risk corrections are automated; ambiguous cases go to manual review. AI never acts unilaterally.

02

CERTIFY

Full Traceability

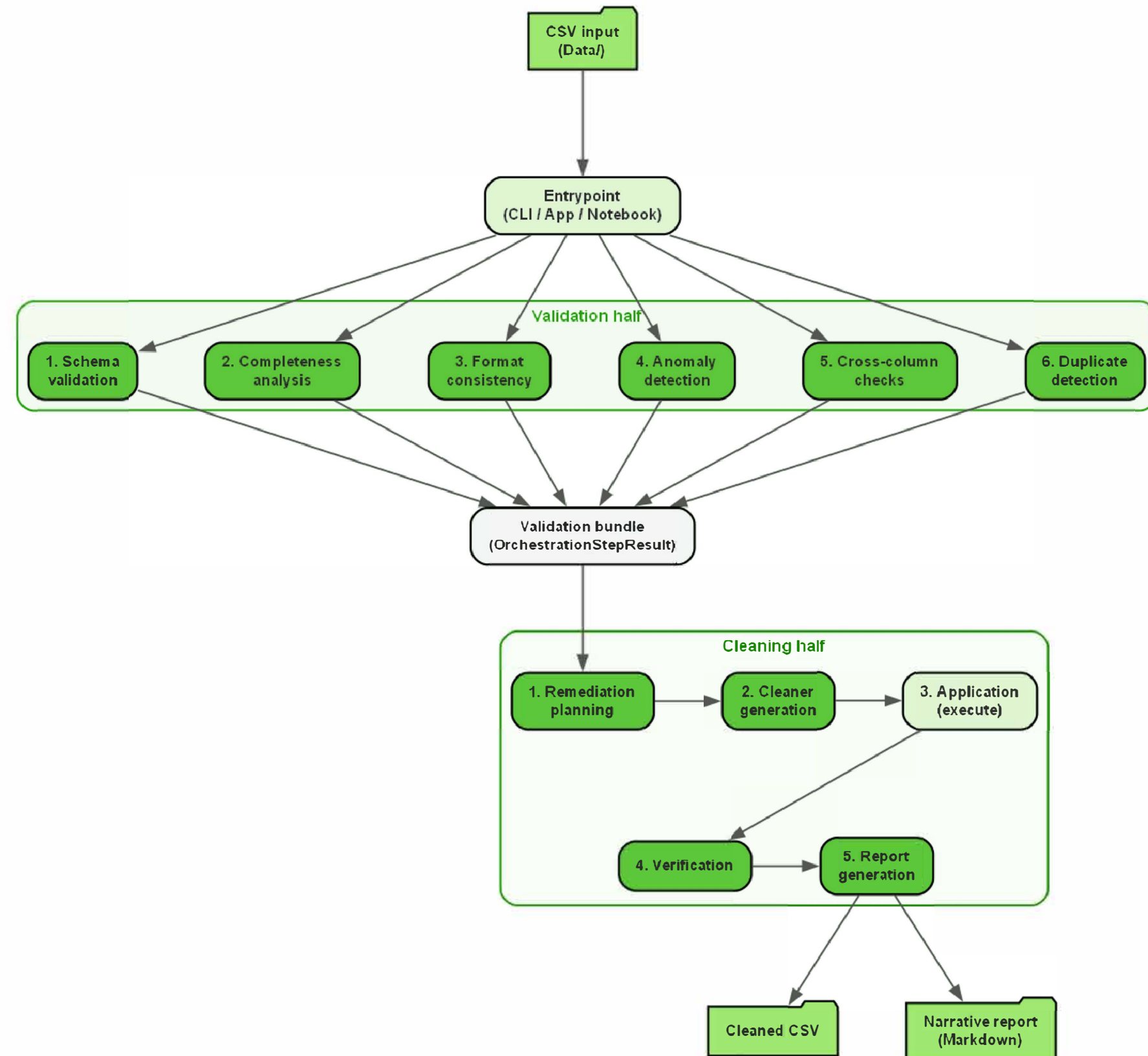
Every finding, action, and verification outcome is stored as a typed Pydantic artifact. The entire process is observable end-to-end via Logfire, from raw CSV to cleaned dataset.

03

How It Works

A clean split between two halves, inspect first, then clean only what can be justified and verified.

- **Validation half**
Six inspection stages build a complete evidence bundle
- **Remediation planning**
Auto-apply for clear defects; manual review for ambiguous cases
- **Cleaner generation**
AI writes targeted functions under a narrow contract
- **Verification**
Post-application re-check is the true success criterion



— PIPELINE STAGE

Validation Half

Six inspection stages that systematically measure every class of data-quality problem — without touching the data.

01

Six Inspection Stages

01 – Schema Validation

What should each column be?

Infers target dtype and canonical pattern from a bounded 5% sample + parse statistics. Flags naming violations.

02 – Completeness Analysis

Where is data truly missing?

Unifies true nulls, empty strings, and placeholder tokens (N/A, ?, unknown) into one missing-like mask per column.

03 – Format Consistency

Is the format uniform?

Shape-based profiling (e.g. 999999, 9999-99) identifies inconsistent minorities against the dominant canonical form.

04 – Anomaly Detection

What looks suspicious?

IQR outliers ($3 \times \text{IQR}$), suspicious negatives, and rare categorical labels, flagged for review, never auto-rewritten.

05 – Cross-Column Checks

Do columns agree with each other?

Row-wise comparisons detect duplicate columns, date-order violations, and period mismatches. Fully deterministic, no LLM.

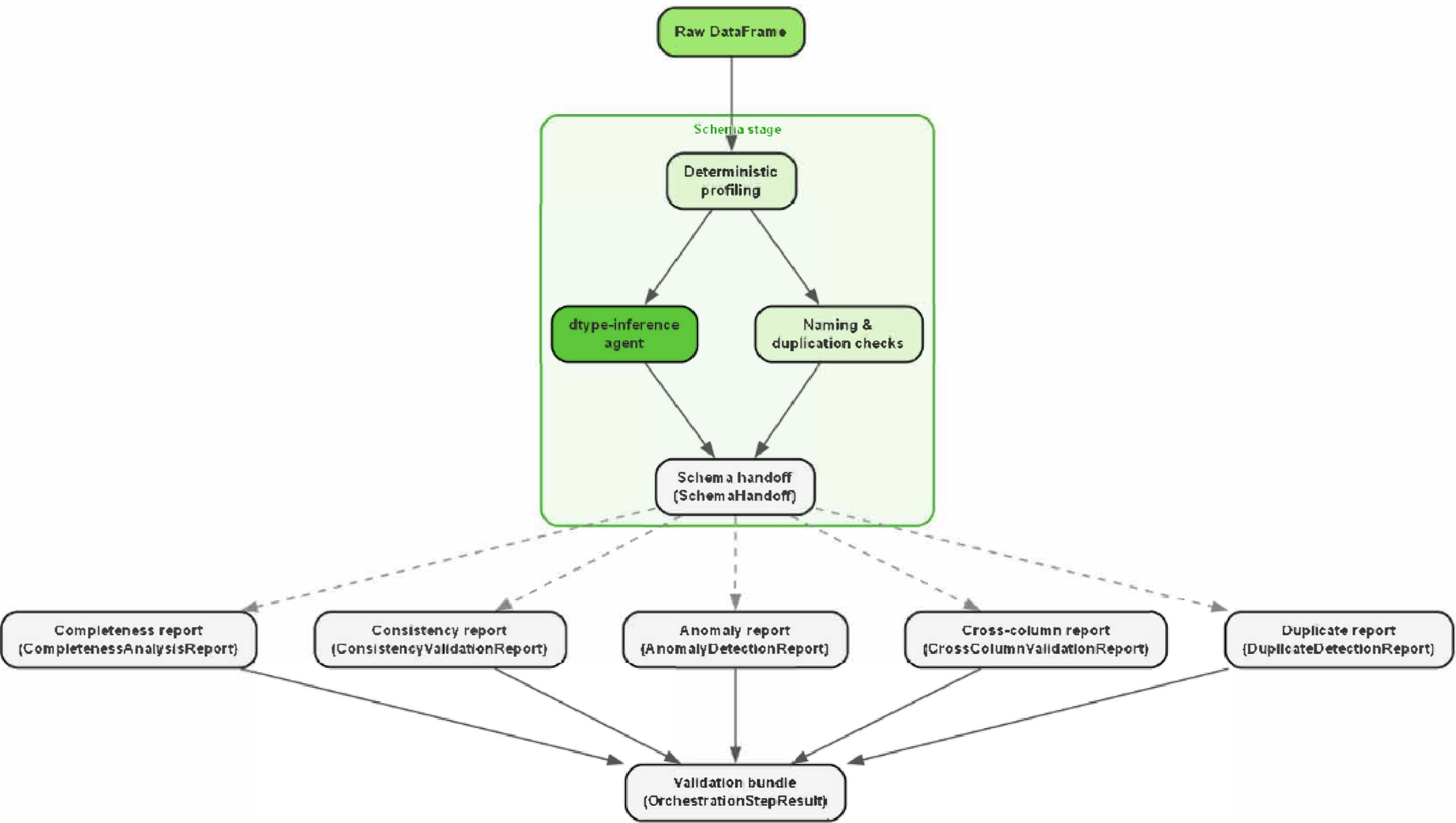
06 – Duplicate Detection

Are rows redundant?

Exact duplicates removed deterministically. Near-duplicates detected via inferred business-key columns.

Deterministic Evidence First

Evidence	Local Python computes parse rates, shapes, placeholder counts, and anomaly bands before any agent is called
Bounded AI	Agents receive distilled artifacts not raw columns. Schema inference uses 5% sample + 500-value cap
Typed output	Every stage emits a Pydantic model (SchemaHandoff, FormatConsistencyFinding, etc.) no free-form text propagation
Bundle	All six outputs merge into a unified ValidationBundle before the cleaning half begins



— PIPELINE STAGE

Cleaning Half

Remediation planning, constrained code generation, host-side validation, and controlled application — AI on a short leash.

The Generator—Critic Loop

Code generation is not one-shot. Every cleaner must survive deterministic host-side validation before touching the real dataset.

01 Remediation Planning

Auto-apply for safe fixes; manual review for ambiguous cases.

02 ColumnCleaningRequest

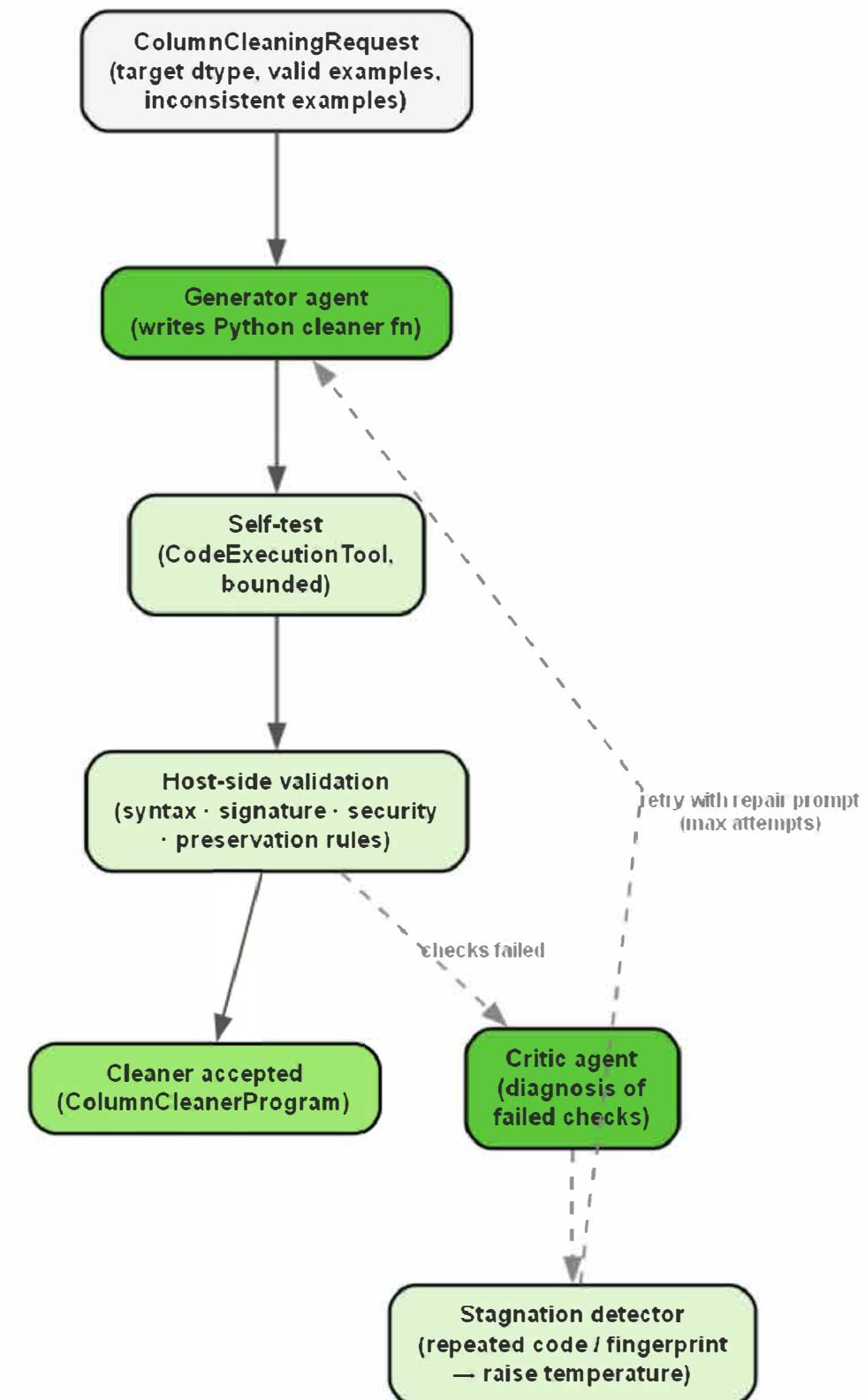
Target dtype, dominant examples, inconsistent examples, preservation rules.

03 Generator + Host Validator

AI writes a self-contained function. Host validates it against representative examples deterministically.

04 Repair Critic

Failures injected back as authoritative context. Temperature raised on stagnation.



— PIPELINE STAGE

Verification & Reporting

Post-application re-check on the real dataset, followed by a structured factual report — then a narrative layer on top.

03

Measurable Outcomes

Safety-first policy: diagnosis is deliberately broader than intervention.

7,543

Raw rows processed

71

Actions applied

6

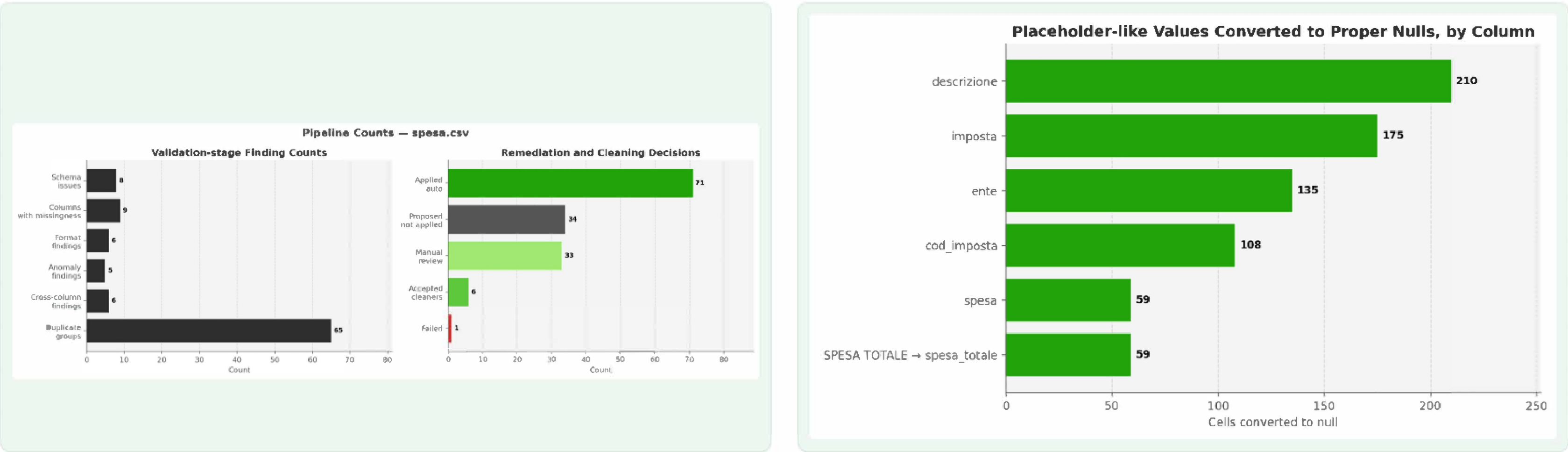
Cleaners accepted & applied

1,943

Rows changed by cleaners

100%

Targeted inconsistencies eliminated



Cost-Efficient by Design

Token usage scales with validated cleaning opportunities, no raw data exposure.

\$0.07

Total model cost, full run

~\$0.03

Cleaner generation (dominant)

\$0.00

Profiling & duplicate checks

t = 0

Model temperature
deterministic agents

AGENT COST BREAKDOWN		
column-cleaner-generator		~\$0.030
narrative-section		~\$0.010
cleaner-repair-critic		<\$0.010
validation + summary agents		~\$0.000

Why so cheap?

Deterministic Python handles all raw profiling, parse rates, shapes, placeholder counts, duplicate detection, without any LLM calls.

The model is invoked only where semantic reasoning is genuinely necessary: dtype inference from bounded samples, generating cleaning functions, and writing the narrative report.

Cost is proportional to the number of validated cleaning opportunities, not to dataset size.

Why This Approach Wins

OPTION A Manual Cleanup

- ✓ Flexible, domain-aware
- ✗ Slow and costly at scale
- ✗ No reproducibility
- ✗ No audit trail
- ✗ Does not scale across recurring datasets

OPTION B Rule-Only System

- ✓ Consistent on known patterns
- ✗ Breaks on novel or messy formats
- ✗ No semantic understanding
- ✗ Constant rule maintenance burden
- ✗ Cannot detect hidden missingness

OPTION C Single AI Prompt

- ✓ Flexible, quick to set up
- ✗ Outputs are unvalidated
- ✗ Non-reproducible by design
- ✗ No governance or explainability
- ✗ Cannot verify improvements

OUR SOLUTION NoiPA Pipeline

- ✓ Adapts to any column format
- ✓ Every fix validated before apply
- ✓ Full end-to-end audit trail
- ✓ AI bounded by deterministic checks
- ✓ Cost scales with cleaning quality

What We **Deliberately** Optimised For

- 01 Safety** Data is not modified unless the correction is clearly defensible and evidence-backed.
- 02 Interpretability** Every stage produces inspectable JSON artifacts. Nothing is a black box.
- 03 Modularity** Validation, planning, generation, application, and verification are fully separated.
- 04 Selective Automation** Routine structural defects are automated; ambiguous or risky cases are escalated.
- 05 Operational Usability** Available via notebook, CLI, and Streamlit app with no extra configuration.

The system was designed for controlled adoption in real organisations, not just for a demo.

What the System Does **Not Try to Solve** Today

Current Limitations

Conservative by design: some issues remain flagged for manual review

Stronger on structured tabular normalisation than free-text semantic correction

Factually wrong but syntactically valid values require business review

Anomaly handling and row-level duplicate drops are intentionally cautious

Why Conservative?

In public-sector contexts, an incorrect automated fix is worse than no fix. Conservative defaults ensure the system builds trust before increasing automation scope.

Scope can be widened over time once trust is established.

The current version prioritises trust and defensibility over aggressive automation.

Roadmap and Next Steps

01 Add configurable intervention policies by dataset or business context.

02 Enable user approval workflows for less conservative changes when speed matters.

03 Expand governance features with human-in-the-loop review and audit exports.

04 Improve support for broader dataset types and more complex remediation scenarios.

We are not proposing fully autonomous data cleaning. We are proposing a **governed data quality copilot** that reduces manual effort while preserving control.

— CONCLUSION —

Thank You.

NoiPA Data Quality Copilot · Reply & LUISS · ML 2025/26

Michele Turco

Mattia Sebastiani

Sofia Bruni